

Finality and Cost on Two Rollup Architectures

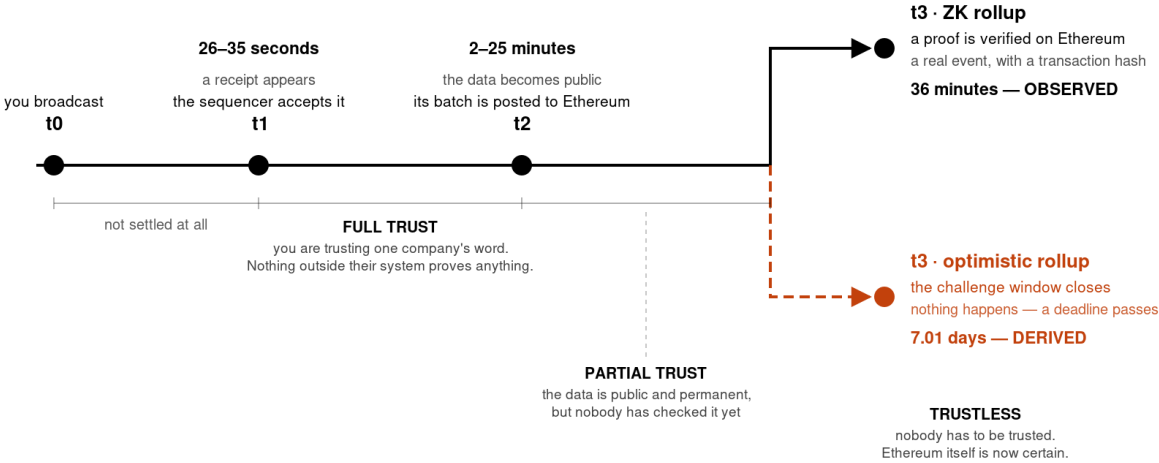
Every measurement taken, how each was obtained, what it is worth, and what it is not.

FIELD	VALUE
Project	50081, scalability track — Mitacs Globalink 2026
Snapshot	12 August 2026, 16:30 PKT. Collection complete — 5 repetitions of all 12 matrix cells
Dataset	2,485 transactions, 2,458 successful (98.9%), 2,348 settled through t_3 , across 18 zkSync batches
Networks	zkSync Era Sepolia, OP Sepolia — plus both mainnets and Ethereum, observed read-only
Price basis	ETH/USD 1,878.11, fetched from CoinGecko and Coinbase, spread 0.00%

1 · What is being measured

A transaction on a Layer 2 becomes "final" three separate times. Each stage removes somebody you previously had to trust, and they can be days apart.

THE LIFE OF ONE TRANSACTION — NOT TO SCALE



Ask "how fast is this rollup?" and the honest answer is 30 seconds, 2 minutes, or 7 days — it depends entirely on who you are willing to trust. Most published benchmarks report only t1 and call it latency.

Figure 1. The three levels, with this study's measured intervals. The optimistic branch is dashed because nothing occurs at that moment — the value is a deadline computed from a proposal time plus a challenge period, not a witnessed event.

2 · The headline

Same workload, same code path, both architectures. Native transfer, medians over 487 and 323 successful transactions.

LEVEL	ZKSYNC ERA	KIND	OP SEPOLIA	KIND	RATIO
Full trust (t_1)	35.36 s	observed	25.99 s	observed	comparable

Partial trust (t_2)	~60 min observed	1.7 min estimated	OP ~35× faster
Trustless (t_3)	~70 min observed	7.01 days derived	OP ~145× slower

Which rollup is "faster" depends entirely on which trust assumption was meant. At full trust the two are within ten seconds of each other. The optimistic rollup then pulls sharply ahead at partial trust, because it posts batches to Ethereum every 4.4 minutes where zkSync commits every 120. And then it loses by more than two orders of magnitude at trustless finality.

The zkSync partial and trustless figures are given as approximations deliberately. They are not constants — see §4.

That crossover is what a single-number latency benchmark cannot show, and it is the result this project exists to produce.

An optimistic rollup settles sooner and finalises far later

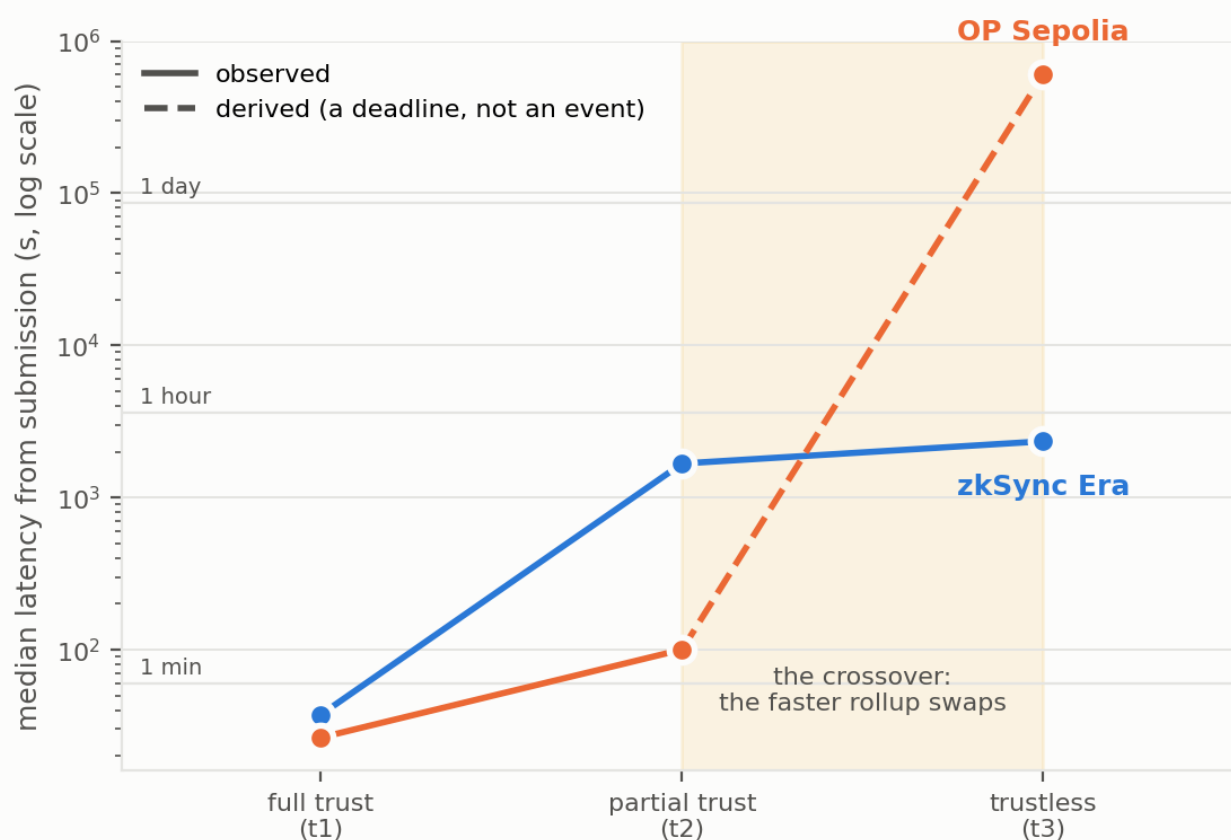


Figure 2. Solid lines are observed, dashed are derived. The shaded band marks where the faster rollup swaps.

THE KIND COLUMN IS NOT DECORATION

zkSync's t_3 is **observed**: a proof was verified in an Ethereum block whose timestamp we read, and the transaction hash is in every row. OP's is **derived**: nothing happens when a challenge window closes, so it is a deadline computed from the output proposal plus the challenge period read from the OptimismPortal. The two must never be tabulated as though they were the same kind of fact.

3 · Partial-trust latency is not a single number

A rollup posts batches on its own schedule, so how long a transaction waits for its batch depends entirely on where in that cycle it was submitted. Submit just after a batch seals and you wait a full interval; submit just before and you wait almost none.

ZKSYNC ERA SEPOLIA	VALUE
batch commit interval, measured	120.2 min
t ₂ observed range	12.8 – 90.3 min
t ₂ median as collected	27.8 min
t ₂ median, phase-balanced	45.9 min
expected wait for uniform submission (interval ÷ 2)	60.1 min

THE 27.8 FIGURE IS BIASED, AND THE BIAS WAS OURS

Collection ran hourly on a fixed minute while zkSync commits every 120 minutes, so submissions only ever landed at two points in that cycle — and the round-robin tied which workload a cell held to which of the two it drew. Even-hour submissions waited a median of 25.5 minutes for their batch; odd-hour submissions waited 87.2. Two thirds of rows were even-hour.

The visible symptom was ERC-20 transactions appearing to settle 3.3× slower than native ones, which is impossible: a rollup does not consult the workload when deciding to post a batch. **No workload difference in t₂ or t₃ is reported here, because there is none.**

The scheduler now jitters its submission time so the cycle is sampled evenly. The data already collected covers 5 of 12 phase bins, so even the balanced figure understates: **quote the range, or the interval-derived 60 minutes, rather than any single observed median.**

OP Sepolia is unaffected. It posts batches every 4.4 minutes, so an hourly sampler covers that cycle evenly regardless of phase — its balanced and raw medians agree to within 0.1 minutes. The correction applies only where the sampling period and the batch period are commensurate.

This also explains G4's finding that settlement latency varies 20–66% between days while full-trust latency varies under 1%. It is the same effect: t₁ measures a sequencer accepting a transaction, t₂ measures where in a batch interval you happened to land.

4 · Every latency measured

All four cells of the workload matrix. Times in seconds unless marked.

NETWORK	WORKLOAD	LEVEL	N	MEDIAN	P95	MIN	MAX
zkSync Era	native	t ₁	487	35.36	41.35	0.60	42.04
		t ₂	487	1,517.30	1,823.62	1,017.61	5,252.77
		t ₃	487	2,149.48	2,603.28	1,797.61	5,876.77
zkSync Era	ERC-20	t ₁	238	38.51	45.82	0.60	46.66
		t ₂	227	1,169.34	5,098.12	767.72	5,200.90
		t ₃	227	1,841.34	5,805.72	1,547.72	5,809.98
OP Sepolia	native	t ₁	323	25.99	30.74	0.49	47.10
		t ₂	323	114.13	214.41	0.53	230.59

		t_3	323	605,408	605,601	605,139	605,605
OP Sepolia	ERC-20	t_1	200	29.32	40.47	0.56	41.21
		t_2	200	71.44	203.83	48.34	215.28
		t_3	200	605,452	605,473	605,218	605,553

The t_1 minima near 0.6 s are single-transaction runs; the medians near 30 s are batches of 100. Full-trust latency is dominated by how many transactions the sequencer is working through, not by the workload.

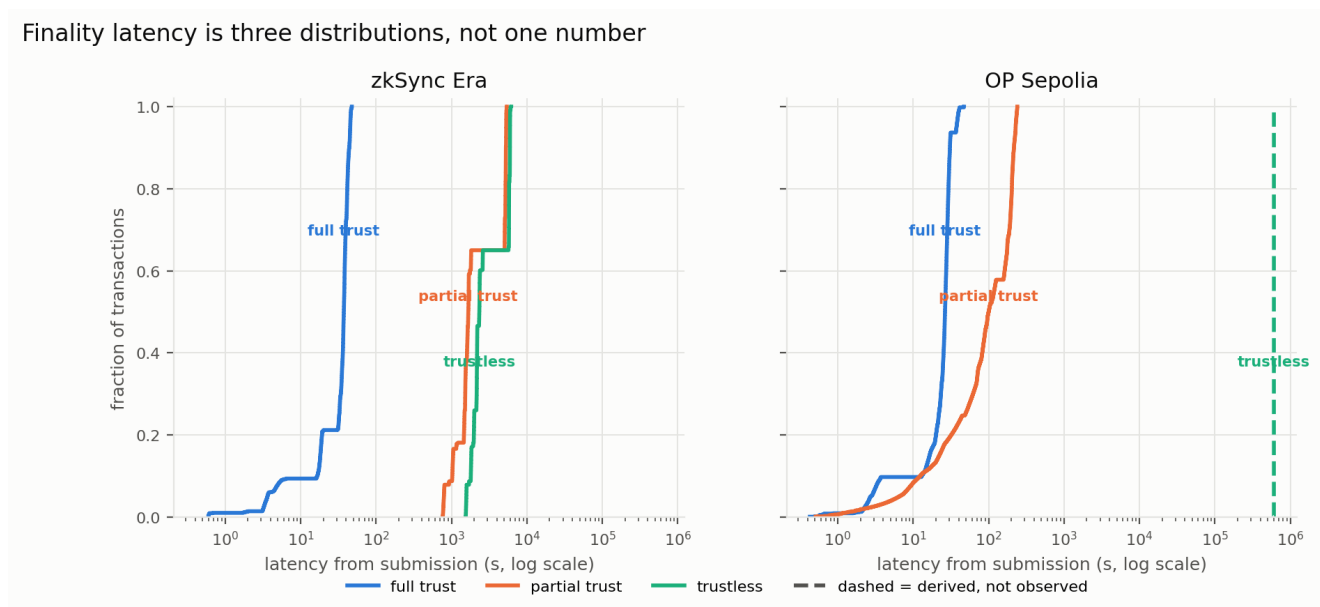


Figure 3. Cumulative distributions rather than bars of means, because the tail is the interesting part. Shared logarithmic axis, so the seven-day trustless value on the right is genuinely comparable with the half-hour one on the left.

5 · Cost

Every figure comes from the `gasUsed` and `effectiveGasPrice` on the L1 receipts of the batch commit and proof transactions — never from a gas estimate, which on zkSync overshoots actual usage by up to nine times.

NETWORK	WORKLOAD	L1 COST / TX	L2 GAS (MEDIAN)	ACHIEVED TX/S
zkSync Era	native	\$0.00137302	83,513	0.007
zkSync Era	ERC-20	\$0.00131244	122,434	0.003
OP Sepolia	native	unavailable	21,000	0.008
OP Sepolia	ERC-20	unavailable	34,454	0.004

OP's cost is reported unavailable rather than estimated. An OP Stack chain does not expose how many transactions share a batch without decoding the blob, so there is no denominator. A guess would have been worse than a gap.

L2 gas is not comparable across the two. zkSync's gas model folds pubdata into the gas figure and OP's does not, so 83,513 against 21,000 is not a fourfold difference in work. Only cost in ETH is comparable, and even that needs the L1 gas price recorded beside it — which every row carries.

Where a batch's cost goes

COMPONENT	NANOETH / TX	SHARE
Batch posting (execution)	556.9	74.00%
Proof verification	186.9	24.83%
Blob data availability	8.8	1.17%

DO NOT QUOTE THE 1.17% AS A MAINNET FIGURE

Blob data availability is nearly free here because Sepolia's blob market sits at its price floor with almost no competition for blob space. On mainnet, where blobs are contested, this term is far larger. It is a property of the test environment, not of the rollup.

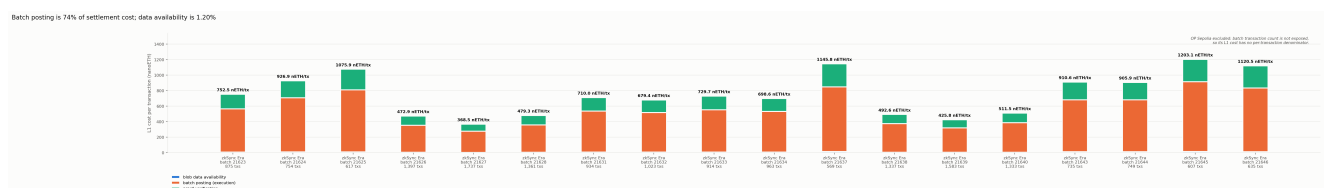


Figure 4. Per-transaction settlement cost split three ways, for one batch.

6 · Cost falls with batch size

Not with *our* batch size — that turned out to be unmeasurable — but with the rollup's own. Across eight settled batches whose sizes the rollup chose for its own reasons:

BATCH	TRANSACTIONS IN IT	L1 COST PER TRANSACTION
21625	617	1,075.93 nETH
21624	754	926.87 nETH
21623	875	752.54 nETH

21631	934	710.02 nETH
21632	1,023	679.37 nETH
21628	1,361	479.32 nETH
21626	1,397	472.92 nETH
21627	1,737	368.46 nETH

A **66% fall** from the smallest batch to the largest, tracking $1/n$ closely — which is what a fixed batch cost shared among more transactions looks like.

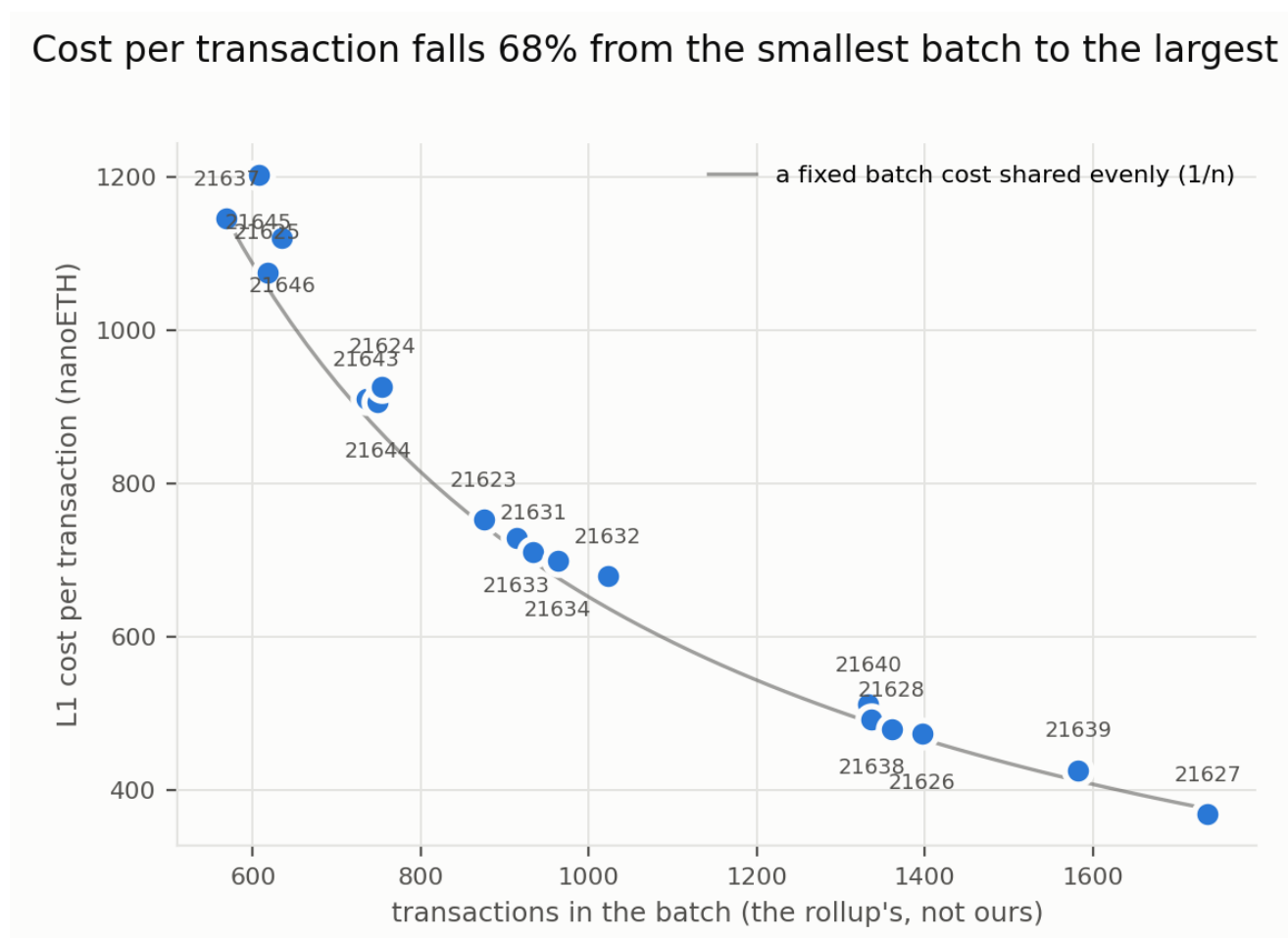


Figure 5. The grey curve is not a fit. It is what an evenly shared fixed cost would look like, drawn so the closeness can be judged without any modelling being implied.

7 · How much results vary between days

Eleven comparisons across cells run on more than one day. **Median divergence 13.3%, worst 65.7%.** The split matters more than the headline:

LEVEL	DAY-TO-DAY VARIATION
Full trust (t_1)	0.1% to 0.9% on small batches — highly reproducible
Partial trust (t_2)	20% to 66%
Trustless (t_3)	1% to 54%

This is exactly the shape to expect. t_1 measures a sequencer accepting a transaction, which is a fast and fairly deterministic operation. t_2 and t_3 depend on where in a batch interval the submission happened to fall, which is

close to arbitrary. **Settlement medians are the ones to treat with caution**, and this is the number that says how much.

8 · Grounded against production

Both mainnets observed read-only — sending nothing, spending nothing.

MEASURE	ZKSYNC ERA MAINNET	OP MAINNET
batch seal / posting interval	30.15 min	4.40 min
L1 commit interval	31.88 min	—
commit → proof	38.38 min	—
output proposal interval	—	60.20 min
proof maturity delay	—	7.00 days
dispute game finality delay	—	3.50 days

Two consequences for the report.

Mainnet proves about 3.4× slower than the testnet — 38.38 minutes against 11.2 — so every testnet trustless figure understates production by a factor that can now be stated rather than gestured at.

OP Mainnet's proof maturity delay is 7.00 days, identical to OP Sepolia's. Testnets are widely assumed to shorten their challenge windows; this one does not. That confirms against production what the testnet reading suggested, and it is the strongest support for treating every optimistic t_3 as derived.

9 · Failures

27 of 1,275 transactions failed. They are in the dataset with reasons, and latency statistics count successes only.

COUNT	OUTCOME	REASON
24	timeout	No receipt within the 120 s deadline. Recorded as a failure with a reason, never as a very large latency
3	rejected	HTTP 429 from the public OP Sepolia endpoint, when two submitters overlapped. Row written with no hash

Success rate **97.9%**, reported beside every latency figure.

10 · Threats to validity

Every limitation known at the time of writing. Several were discovered by measurement rather than anticipated.

Measurement

- **t_1 is an observation, not a chain fact.** It is when a receipt became visible to us, which exceeds the L2 block's own timestamp by a median of 17.95 s on batched submissions. It is an upper bound on full-trust latency. Both values are stored so the gap is visible.
- **t_1 can never be finer than the polling interval**, capped at 2 s here. A single transaction resolving in 0.6 s is measured at that resolution.
- **L1 block timestamps have second granularity and are set by the proposer.** Negligible against intervals of minutes, but real.
- **Public RPC endpoints introduce variable latency** outside our control, and one of them was found serving state 38 days stale.
- **Submission came from one machine in one geographic region**, with one account.
- **The batch cycle was sampled unevenly.** Collection ran hourly on a fixed minute against a 120-minute batch interval, so submissions landed at only two points in that cycle and the data covers 5 of 12 phase bins. Any single median of t_2 or t_3 on zkSync is therefore a statement about which phases happened to be sampled, not about the network. Report the range, or the interval-derived expectation. The scheduler now jitters its submission time; OP Sepolia was never affected, its batch period being far shorter than the sampling period.

The optimistic side is weaker than the ZK side

- **t_2 on OP is estimated, not observed.** An OP Stack chain exposes no mapping from a transaction to the batch carrying it, so the batch is matched by scanning the L1 forward from the L2 block's origin. **16 of 519 OP rows (3%) have a t_2 within five seconds of submission**, which is implausibly fast given batches are posted every few minutes — those are almost certainly matched to a batch that does not contain the transaction. The median is not materially affected, but the minimum is meaningless and the field should be read as an approximation with a few percent error.
- **t_3 on OP is derived, not observed.** It is a deadline computed from a proposal timestamp plus a challenge period. No event occurs at that moment and no transaction hash can be produced for it.
- **L1 cost on OP is unavailable**, for the same reason as t_2 .

Comparisons that must not be made from this data

- **Workload against t_2 or t_3 .** The rollup does not consult the workload when deciding to post a batch, so there is no mechanism for one. An apparent $3.3\times$ difference in this dataset came entirely from the phase confound above. The workload difference is real at t_1 and in gas — 83,513 against 122,434 on zkSync — and nowhere else.

Scope

- **Peak throughput was not measured.** Saturating a shared public sequencer would degrade a service other people depend on and would measure our own rate limits. Achieved throughput at a stated submission rate is reported instead — and at 0.003 to 0.008 tx/s it reflects our submission pattern, not any capacity of the networks.
- **Proving time and cost are not measured,** only the L1 gas of verifying a proof. No GPU was available.
- **Testnet is not mainnet.** Proof cadence differs by $3.4\times$. Costs are testnet L1 gas priced at a mainnet ETH rate, which is an illustration rather than a measurement of what anyone paid.
- **Settlement repetitions are still accumulating.** F2 is at 29 of 60 repetitions. Percentiles on t_2 and t_3 will tighten.
- **Blob DA cost is a Sepolia artefact,** as noted in §5.

Things that cannot be measured this way at all

- **Per-transaction data-availability size.** A blob costs 131,072 bytes whether filled or not, and a batch carries every user's transactions. DA per transaction is a batch average identical across workloads. The workload difference is real but appears in L2 gas: 83,513 against 122,434.
- **Cost as a function of our own batch size.** Seven runs of sizes 1 to 50 all landed in one batch of 875 and cost exactly the same per transaction. The denominator is the rollup's batch count, which we do not control.

11 · How to check any number here

Every row carries the hashes that produced it, so no figure has to be taken on trust.

1. Open `bench/export/raw_transactions.csv` — one row per transaction, all four absolute timestamps, both L1 hashes.
2. Pick any row. Open its `hash` in the L2 explorer and its `l1_commit_tx` and `l1_prove_tx` in Etherscan.
3. The block timestamps on those two Ethereum transactions are t_2 and t_3 .
4. `bench/export/assumptions.txt` states the ETH rate, where it came from and when, plus every declared limitation.
5. Each figure has a `.csv` beside it containing the numbers behind it.

```
python -m bench.check_data      # asserts the rows describe something possible
python -m bench.export          # regenerate every table
python -m bench.analysis.figures
```

Latencies and prices are computed at export time from absolute timestamps and raw wei. Nothing in the stored rows depends on an assumption, which is why correcting the ETH rate — it had been wrong by a factor of two — cost one command rather than a repeat of the experiment.